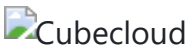


Cubecloud Agentic-OS

English · 简体中文 · 日本語 · 한국어

> **A local-first agent desktop and operating model for teams that want portability, auditability, and lower AI operating cost.** Cubecloud gives builders one control plane for runtimes, providers, skills, memory, schedules, and optional code intelligence - without turning the user's machine into a thin client for a hosted wrapper.

Cubecloud Agentic-OS is the monorepo for the **Cubecloud Agent Desktop** and the operating model around it. The desktop binary lives in `agent-desktop/`. The Cubecloud-original control plane, prelaunch bundle, and developer-time skill ecosystem live in `apps/desktop-shell/`, `packages/platform-core/`, and `.agents/`.

The short version:

- Keep prompts, skills, memory, and runtime choices in files, SQLite, and explicit local contracts.
- Use local runtimes for routine loops and reserve paid remote inference for the turns that deserve it.
- Swap runtimes and providers without rewriting the operating model.
- Give developers and operators one desktop instead of a pile of CLIs, browser tabs, and vendor dashboards.

Preview

A curated subset of the desktop surfaces — what a first-time reader should see before reading the architecture sections. Every image is a full-page capture from the current desktop build. The full 22-image gallery (onboarding, runtime discovery, and every major operator surface exposed in the sidebar) lives at `agent-desktop/README.md`.

Welcome & first-run	**Runtime detection**
	

Chat Chat	**Profiles & agents** Agents
Gateway (16 platforms) Gateway	**Settings & control** Settings

Why teams adopt Cubecloud

Cubecloud is for teams that want the convenience of a desktop product without giving up the control surface of a self-directed stack.

Outcome	How Cubecloud gets there
Faster first useful session	A prelaunch bundle ships memory seeds, disabled harnesses, a disabled schedule, and a starter kanban board so the first launch is not an empty shell.
Lower operating cost	Local-first lanes handle drafting, retrieval, orchestration, and iteration on already-paid hardware; remote frontier models stay optional.
Lower vendor risk	Runtime choice and provider choice are separate decisions, so a model or vendor change is a reconfiguration event instead of a rewrite event.
More reproducible operator workflows	Skills, schedules, provider definitions, and state live in inspectable files, SQLite, and explicit IPC surfaces rather than in a hosted black box.
Cleaner procurement and legal review	Cubecloud-original work is available under AGPL-3.0-or-later, Apache-2.0, or MIT, while inherited framework code remains MIT and documented path-by-path.

Why local-first wins

Local-first here is not a marketing adjective. It changes where the control plane lives, how costs accumulate, and how much of the system a team can inspect when something goes wrong.

Decision area	Hosted wrapper default	Cubecloud local-first model
Control plane	Vendor account, vendor UI, vendor retention loop	Native desktop under the local user's control
Cost shape	Seat fee + token fee + wrapper economics	Local hardware for routine work, remote spend only where it adds value
State and provenance	History and orchestration live primarily in the hosted product	Prompts, skills, schedules, and memory remain inspectable and reproducible
Runtime change	Often means switching products or accepting vendor abstraction limits	Runtime picker keeps the operating surface stable while runtimes evolve
Provider change	Usually vendor-first, sometimes BYOK second	Provider layer is explicit and separate from runtime choice
Failure recovery	Wait for vendor fixes or inspect partial logs	Inspect local state, logs, config, and IPC boundaries directly

BYOK is a procurement control. Local-first is an operating model. BYOK changes whose bill arrives. Local-first changes how much billable remote work the workflow requires in the first place.

Who this is for

Cubecloud is a strong fit for:

- Teams building internal agent tooling that still needs security review, provenance, and rollback paths.
- Consultancies and platform teams that need to ship per-client agent stacks without coupling every deployment to the same hosted wrapper.
- Developers who want desktop convenience without giving up local runtime control.
- Cost-sensitive operators who want to keep iterative work local and use remote models selectively.

It is a weak fit if you want a pure browser product, a hosted SaaS control plane, or a model vendor to own your runtime lifecycle for you.

What ships in this repo

This monorepo is broader than the desktop binary alone.

- `agent-desktop/` is the full Electron desktop that ships to end users.
- `apps/desktop-shell/` is the Cubecloud-original state and control-plane workspace.
- `packages/platform-core/` holds shared TypeScript contracts.
- `.agents/skills/` contains 34 first-class open-source skills adapted from 7 upstream repos and mirrored to `~/.agents/skills/`.
- `.github/skills/headroom-workflow/` is a repo-authored Copilot / VS Code workflow layer for the optional Headroom context-compression proxy. The standalone install/use guide lives at `docs/agent-skills-bundle/HEADROOM.md`.
- `docs/` holds the handbook, threat model, runtime plans, legal policies, and transition history.

On first launch, the desktop gives the user:

- A native Electron desktop built with React 19, i18next, Vite, and electron-builder.
- A multi-runtime picker: Hermes today, with OpenClaw and IronClaw planned as additional lanes.
- A provider layer that can talk to local providers such as Ollama, vLLM, and llama.cpp or remote OpenAI-compatible APIs.
- Three user-visible first-launch skills: `cubecloud-persona`, `cubecloud-onboarding`, and `cubegraph-code-intel`.
- A prelaunch operating context with seeded memories, harness placeholders, a schedule placeholder, and a starter kanban board.

- Optional CodeGraph and EverOS integrations that are user-initiated, not silently installed.

What it does **not** do:

- It is **not** a model server; it consumes runtime and provider protocols instead of bundling inference.
- It is **not** a hosted IDE; the desktop is the local control surface.
- It is **not** a single-vendor wrapper; runtime choice, provider choice, and skill assets stay portable.

Market position

Cubecloud is not trying to be the best cloud copilot, the best vendor CLI, or the best demo app template. It is aimed at a different buyer: the team that cares most about control, portability, and unit economics.

Market option	Strong at	Constraint	Cubecloud position
Cloud IDE copilots such as Cursor or GitHub Copilot agents	Fast hosted coding loops and deep IDE assistance	Cloud-default state, seat economics, and a more vendor-centric control plane	Cubecloud centers a local desktop operator and keeps runtimes, providers, and skill assets swappable
Single-vendor CLIs such as Claude Code or Codex CLI	Strong terminal-native loops for a specific vendor stack	Terminal-first UX and narrower runtime portability	Cubecloud offers a GUI-first control plane with a portable runtime and provider model
Reference repos and quickstarts	Learning, demos, and fast prototyping	Not an opinionated operating surface or long-lived operator workflow	Cubecloud ships a real desktop workflow, a handbook, seeded context, and a documented provenance posture

Market option	Strong at	Constraint	Cubecloud position
BYOK wrappers	Easier procurement conversations	Often still wrapper-seat economics on top of token costs	Cubecloud uses local-first design to reduce how much of the workflow needs paid remote inference at all

The strategic point is simple: many competitors optimize for **vendor depth**. Cubecloud optimizes for **operator control**.

Built for production-minded teams

Production-ready in Cubecloud's sense does not mean "hosted SaaS with a sales dashboard." It means the core operating surfaces are explicit, inspectable, and replaceable.

- **Explicit trust boundary.** The renderer is sandboxed, IPC channels are explicit, outbound networking is opt-in, and inbound networking is opt-in on a user-supplied port. See `SECURITY.md` and `THREAT_MODEL.md`.
- **Predictable state.** Profiles, sessions, provider definitions, memories, schedules, and kanban state live in durable local state rather than in an opaque hosted workflow layer.
- **Replaceable dependencies.** Runtime choice and provider choice are separated so teams can migrate, stage, or roll back without collapsing the whole user workflow.
- **Optional sidecars stay optional.** CodeGraph and EverOS expand the system when needed, but they do not become mandatory hidden dependencies.
- **Versioned methodology.** The 34-skill ecosystem is documented, provenance-tracked, and supported by a red-baseline discipline inherited from the upstream skill process.
- **Clear legal surface.** The repo documents path-level provenance, trademark posture, commercial-relicensing policy, and the inherited-framework carve-out in one place. See `BRANDING_AND_LICENSE.md` and `docs/legal/`.

That is the enterprise story: not "trust us," but "inspect the stack."

Architecture at a glance

The desktop experience is built from three cooperating layers:

Core runtime layer

- **State layer** - `apps/desktop-shell/src/main/agentControlPlane.ts` owns profiles, sessions, models, providers, skills, memory, schedules, and kanban state.
- **Runtime orchestration** - `docs/RUNTIME_ORCHESTRATION_PLAN.md` describes Hermes as the current lane and OpenClaw / IronClaw as the next lanes.
- **Provider layer** - `apps/desktop-shell/src/main/providerDiscovery.ts` keeps model-provider selection separate from runtime selection.
- **Skills harness** - `agent-desktop/src/main/skills-harness.ts` applies the skill layer around outgoing requests.

Integrated support surfaces (optional, user-initiated)

- **CodeGraph surface** - `docs/CODEGRAPH-RUNTIME.md` explains the optional semantic code-intelligence path.
- **EverOS sidecar** - `docs/EVEROS-SIDECAR.md` explains the optional memory and harness sidecar lifecycle.

User-managed third-party applications

- The desktop can connect to tools the operator already uses — for example Open WebUI, OpenCode, Warp ADE, VS Code, Ollama, LM-Studio, Odysseus, ComfyUI, or Open Design. These are not bundled, not required, and can be added or removed by the user.

Conceptual model (V2.10.35)

Cubecloud's local surfaces map cleanly to a small **object + action** model that is conceptually similar to what Palantir's Foundry calls an "ontology" for the enterprise, but **scoped down to one operator's desk**:

- **Objects (nouns)** are the durable things the agent operates on: profiles, sessions, models, providers, skills, memories, tools, schedules, and kanban tasks. They are persisted as explicit JSON registries under the user's control, not as opaque blobs in a hosted workflow engine.
- **Actions (verbs)** are the things the agent does to those objects: dispatch a chat turn, run a scheduled prompt, commit a learned proposal, apply a CodeGraph query result, or revert an `AGENTS.md` edit. The dispatch ledger in `agentControlPlane.ts` is the action log; the `headroom learn --apply` review flow is the **branch-and-review** gate that prevents AI from writing to the ontology without an explicit human action.
- **Operator scope.** Foundry's ontology is a digital twin of an organization; Cubecloud's is a digital twin of a single desk. This is a deliberate scale difference, not a missing feature. The

same noun/verb structure scales down to a prosumer and small- business operator without forcing them to pay for a multi-tenant platform they do not need.

The point of stating the model explicitly is so a new contributor can map any future feature to either "this is a new object" or "this is a new action" and place it in the right layer of the architecture. The implementation still uses hand-rolled JSON registries; a future hardening pass can collapse them into a single typed registry without changing the conceptual contract.

Optional: Headroom context compression

The desktop's runtime is the user's choice; the cost of using it is also the user's choice. As an additive layer, Cubecloud ships a repo-authored workflow skill at `.github/skills/headroom-workflow/` that guides a Copilot / VS Code session through the `headroom-ai` context-compression proxy when the local token pressure is high (large tool logs, long chat histories, big CodeGraph bundles). The full install path for non-repo Copilot sessions —including the `headroom proxy`, `headroom mcp install`, and `headroom wrap copilot` modes —lives at `docs/agent-skills-bundle/HEADROOM.md`. A one-command helper for mirroring the workflow skill into the user-global Copilot skills directory is at `docs/agent-skills-bundle/install-headroom-workflow.cmd`. Headroom is **never required**: the desktop is fully functional without it.

Optional: Graphify for repo-local coding support

If you want a local knowledge-graph assist while coding in this repo, you can run Graphify as a **developer-side workflow**. This is not an agent-desktop runtime integration and does not change shipping product behavior.

- `npm run graphify:repo` builds a deep graph for the current repo.
- `npm run graphify:update` incrementally refreshes changed files.
- `npm run graphify:query -- "<question>"` runs graph-based queries.

Graphify artifacts are local-only and ignored by git (`graphify-out/`, `.review-graphify/`). If the CLI is not installed, use: `python -m pip install graphifyy`.

Where to start

- **New contributor:** read `docs/HANDBOOK.md` sections 1, 2, 3, and 5.
- **Evaluator of the shipped desktop:** read `agent-desktop/README.md`, then `docs/HANDBOOK.md` sections 1, 3, and 10.

- **Reviewer or release owner:** read `docs/HANDBOOK.md` sections 1, 3, 4, 6, 9, 10, and 11 in that order.

Repository layout

```
cubecloud-agentic-os/
├── README.md                    this monorepo README
├── LICENSE                      Cubecloud-original work: AGPL-3.0-or-
later / Apache-2.0 / MIT
├── NOTICE                       third-party attribution catalog
├── BRANDING_AND_LICENSE.md      licensing, provenance, and transition
history
├── CONTRIBUTING.md              DCO 1.1 contribution contract
├── SECURITY.md                  security policy and reporting
├── THREAT_MODEL.md              local-first threat model
├── README.i18n.md               translation inventory manifest
├── .agents/                     34 open-source skills mirrored to
~/.agents/skills/
├── .github/                     agent instructions, workflow skills, and
automation
├── apps/
│   └── desktop-shell/           Cubecloud-original control plane
workspace
├── packages/
│   └── platform-core/           shared TypeScript contracts
├── docs/
│   ├── HANDBOOK.md              master handbook
│   ├── RETIRED_AND_LEGACY.md    live / mirror / scratch-pad map
│   ├── handbook/                long-form architecture, development, and
operations docs
│   └── legal/                   EULA, trademark, and commercial
licensing policies
├── scripts/
│   ├── sync-docs.ps1            hardlink and junction regeneration
```

```
│   └── v2.10.20-readme-combined-pdf.cjs
└── agent-desktop/            the shipped Electron desktop
```

License

Cubecloud-original work is available under **AGPL-3.0-or-later, Apache-2.0, or MIT**. AGPL-3.0-or-later is the primary license. Apache-2.0 and MIT are compatibility options for downstream consumers whose house policy already centers one of those licenses. Inherited `hermes-desktop` framework code remains MIT under its upstream terms.

See `LICENSE`, `NOTICE`, `BRANDING_AND_LICENSE.md`, and `docs/legal/` for the per-path breakdown.

Contributing

Inbound contributions follow the **DCO 1.1** sign-off model. Every commit must include a `Signed-off-by:` line. See `CONTRIBUTING.md` for the contract.

The skills layer is a primary contributor workflow surface. A new skill typically goes through `gbrain-skillify`, `ecc-skill-scout`, `po-write-a-skill`, and `sp-write-skill`, with a red-baseline test to prove the behavior is worth keeping.

If you find a bug or have a feature request, file an issue. For security issues, follow `SECURITY.md` and do not post secrets, API keys, or private logs in public issues.

Translations

The monorepo now ships:

- Simplified Chinese translations for `README.zh-CN.md`, `CONTRIBUTING.zh-CN.md`, `SECURITY.zh-CN.md`, `THREAT_MODEL.zh-CN.md`, `docs/HANDBOOK.zh-CN.md`, `docs/handbook/` leaf docs, and `docs/RETIRED_AND_LEGACY.zh-CN.md`.
- Japanese and Korean outer README starting points at `README.ja-JP.md` and `README.ko-KR.md`.

The translation inventory lives in `README.i18n.md`. The combined English + Simplified Chinese README PDF lives at `docs/Cubecloud-README-en-zh.pdf`.

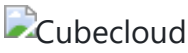
Binary-facing translations still live under `agent-desktop/`. If you want to polish the current translations or extend the monorepo to additional languages, follow the workflow in

`README.i18n.md`.

English → Simplified Chinese

中文版本 · Simplified Chinese

The same document follows, translated to Simplified Chinese (zh-CN).



Cubecloud Agentic-OS 中文文档（zh-CN）

English · **简体中文** · 日本語 · 한국어

> **为追求可移植性、可审计性与更低 AI 运行成本的团队打造的本地优先智能体桌面与运作模型。** Cubecloud 将运行时、提供者、技能、记忆、计划任务与可选代码智能 统一到一个控制面中，而不是让用户的设备沦为某个托管包装层的薄客户端。

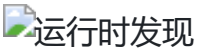
Cubecloud Agentic-OS 是 **Cubecloud Agent Desktop** 及其运作模型的单仓代码库。 桌面端二进制文件位于 `agent-desktop/`。 Cubecloud 原创的控制平面、预启动上下文与开发期技能生态位于 `apps/desktop-shell/`、 `packages/platform-core/` 与 `.agents/`。

四句话讲清楚：

- 提示词、技能、记忆与运行时选择，沉淀为文件、SQLite 与显式本地契约中的可管理资产。
- 高频迭代尽量留在本地，付费远程推理只留给真正高价值的回合。
- 切换运行时和提供者，不需要重写整套操作模型。
- 一个桌面控制面，取代分散的 CLI、浏览器标签页与厂商后台。

预览

下面是桌面关键表面的精选图集——首次阅读者在阅读架构章节前应该看到的内容。 每一张都是当前桌面构建的全页截图。完整的 22 张图廊（覆盖引导、运行时发现、 以及侧边栏中所有主要操作面）位于 `agent-desktop/README.zh-CN.md`。

欢迎 & 首次启动	**运行时发现**
	

对话 	档案与代理 
消息网关（16 个平台） 	设置与控制 

为什么团队会采用 Cubecloud

Cubecloud 面向那些既想要桌面产品的便利，又不愿放弃对技术栈控制权的团队。

结果	Cubecloud 如何实现
更快进入第一个有价值的会话	预启动包自带记忆种子、默认关闭的 harness、默认关闭的计划任务和入门看板，首次启动不是空壳。
更低运行成本	本地优先链路承担起草、检索、编排与迭代，远程前沿模型保持可选。
更低厂商锁定风险	运行时选择与提供者选择是两件事，模型或厂商变化是重配置，不是重写系统。
更可复现的操作流程	技能、计划任务、提供者定义与状态保存在可检查的文件、SQLite 与显式 IPC 表面中，而不是托管黑盒。
更容易通过采购与法务审查	Cubecloud 原创部分提供 AGPL-3.0-or-later、Apache-2.0、MIT 三选一；继承框架保持 MIT，路径级来源清晰可查。

为什么本地优先更有优势

这里的"本地优先"不是营销词，而是对控制面归属、成本结构与故障可检查性的明确选择。

决策维度	托管包装层默认做法	Cubecloud 的本地优先模型
控制面归属	厂商账号、厂商 UI、厂商留存回路	本地用户掌控的原生桌面

决策维度	托管包装层默认做法	Cubecloud 的本地优先模型
成本结构	席位费 + token 费 + 包装层经济模型	日常工作尽量走本地硬件，远程成本只在确实增值时发生
状态与来源	历史记录与编排状态主要留在托管产品中	提示词、技能、计划任务与记忆均可检查、可复现
运行时切换	往往意味着换产品，或接受厂商抽象层的限制	运行时选择器保持操作界面稳定，同时允许底层运行时演化
提供者切换	通常以厂商优先，BYOK 只是补充	提供者层是显式的，且与运行时层解耦
故障恢复	等待厂商修复或查看有限日志	直接检查本地状态、日志、配置与 IPC 边界

BYOK 更像采购控制，本地优先才是运作模型。 BYOK 改变的是账单归属；本地优先改变的是这套流程本身需要多少远程计费工作。

适合谁

Cubecloud 特别适合以下几类团队与操作者：

- 需要通过安全审查、来源审查与回滚审查的内部智能体工具团队。
- 需要为不同客户交付不同智能体栈，又不想把每次部署都绑定到同一个托管包装层的咨询团队与平台团队。
- 想要桌面便利，但不愿放弃本地运行时控制权的开发者。
- 希望把高频迭代工作留在本地、只在必要时调用远程模型的成本敏感型操作者。

如果你需要的是纯浏览器产品、托管 SaaS 控制面，或者希望模型厂商替你接管整个运行时生命周期，Cubecloud 不是最合适的选择。

这个仓库实际交付什么

这个单仓交付的内容远不止一个桌面端二进制文件。

- `agent-desktop/` 是面向终端用户交付的完整 Electron 桌面端。

- `apps/desktop-shell/` 是 Cubecloud 原创的状态层与控制平面工作区。
- `packages/platform-core/` 保存共享 TypeScript 契约。
- `.agents/skills/` 包含 34 个旗舰级开源技能，来自 7 个上游仓库，镜像到 `~/.agents/skills/`。
- `.github/skills/headroom-workflow/` 是仓库自带的 Copilot / VS Code 工作流层，对接可选的 Headroom 上下文压缩代理。完整安装 / 使用指南位于 `docs/agent-skills-bundle/HEADROOM.md`。
- `docs/` 保存手册、威胁模型、运行时规划、法律政策与过渡历史。

用户第一次启动桌面端时，会得到：

- 一个使用 React 19、i18next、Vite 与 electron-builder 构建的原生 Electron 桌面端。
- 一个多运行时选择器：当前为 Hermes，后续规划加入 OpenClaw 与 IronClaw。
- 一个与运行时层分离的提供者层，可连接 Ollama、vLLM、llama.cpp 等本地提供者，也可连接 OpenAI 兼容的远程 API。
- 3 个首次启动即对用户可见的技能：`cubecloud-persona`、`cubecloud-onboarding`、`cubegraph-code-intel`。
- 一套预启动操作上下文，内含记忆种子、harness 占位项、计划任务占位项与入门看板。
- 用户主动启用的可选 CodeGraph 与 EverOS 集成，而非静默自动安装的隐藏依赖。

不做的事情：

- **不是**模型服务器——它消费运行时与提供者协议，而不是自带推理。
- **不是**托管 IDE——桌面端是本地控制面。
- **不是**单厂商包装层——运行时选择、提供者选择与技能资产均保持可移植。

市场定位

Cubecloud 并不是要成为"最好的云端 Copilot"、"最强的单厂商 CLI"或"最轻的演示模板"。它瞄准的是另一类用户：最在意控制权、可移植性与单位经济的人。

市场选项	擅长之处	约束	Cubecloud 的位置
Cursor、GitHub Copilot agents 等云端 IDE Copilot	托管式编码循环快，IDE 集成深	状态默认云端，席位经济模型更重，控制面也更偏厂商	Cubecloud 把本地桌面操作者放在中心，让运行时、提供者与技能资产保持可替换

市场选项	擅长之处	约束	Cubecloud 的位置
Claude Code、Codex CLI 等单厂商 CLI	特定厂商栈上的终端体验很好	终端优先，运行时可移植性更窄	Cubecloud 提供 GUI 优先的控制面，以及更可迁移的运行时 / 提供者模型
参考仓库与 quickstart	学习与演示起步快	缺少长期操作面的约束与工作流	Cubecloud 交付真实的桌面工作流、手册、预置上下文，以及可审计的来源说明
BYOK 包装层	更容易与采购沟通	常常仍叠加包装层席位成本与 token 成本	Cubecloud 用本地优先设计，减少整条流程对付费远程推理的依赖

核心战略很简单：很多竞品优化的是**厂商深度**，Cubecloud 优化的是**操作者控制权**。

面向生产环境的姿态

Cubecloud 所说的"面向生产"，并不是"托管 SaaS 加销售后台"，而是指核心操作面足够显式、可检查、可替换。

- **显式信任边界** — 渲染进程沙箱化，IPC 通道显式定义，出站网络默认需启用，入站网络默认需在用户指定端口上启用。参见 `SECURITY.md` 与 `THREAT_MODEL.md`。
- **可预测状态** — 人设、会话、提供者定义、记忆、计划任务与看板状态均保存在持久化的本地状态中，而非托管黑盒流程层。
- **依赖可替换** — 运行时选择与提供者选择分离，团队可以迁移、灰度或回滚，而不会让整个用户工作流一起失效。
- **可选 sidecar 依然保持可选** — CodeGraph 与 EverOS 在需要时扩展系统，但不会变成必须存在的隐藏平台依赖。
- **方法论已版本化** — 34 技能生态有文档、有来源跟踪，并继承了上游技能流程中的 red-baseline 验证纪律。
- **法律表面清晰** — 仓库在一个地方说明路径级来源、商标姿态、商业重许可政策以及继承框架的 MIT 剖分。参见 `BRANDING_AND_LICENSE.md` 与 `docs/legal/`。

这就是它面向企业团队的价值主张：不是"请相信我们"，而是"请检查这套栈"。

架构一览

桌面体验由三个相互协作的层构成：

核心运行时层

- **状态层** — `apps/desktop-shell/src/main/agentControlPlane.ts` 负责人设、会话、模型、提供者、技能、记忆、计划任务与看板状态。
- **运行时编排** — `docs/RUNTIME_ORCHESTRATION_PLAN.md` 描述了当前 Hermes 主道与后续 OpenClaw / IronClaw 主道。
- **提供者层** — `apps/desktop-shell/src/main/providerDiscovery.ts` 让模型提供者选择与运行时选择解耦。
- **技能 harness** — `agent-desktop/src/main/skills-harness.ts` 在出站请求外层应用技能系统。

集成支持面（可选，由用户主动启用）

- **CodeGraph 接入面** — `docs/CODEGRAPH-RUNTIME.md` 说明可选的语义代码智能路径。
- **EverOS sidecar 辅助** — `docs/EVEROS-SIDECAR.md` 说明可选记忆与 harness sidecar 的生命周期。

用户管理的第三方应用

- 桌面可以连接操作者已经在使用的工具，例如 Open WebUI、OpenCode、Warp ADE、VS Code、Ollama、LM-Studio、Odysseus、ComfyUI 或 Open Design。这些应用不捆绑、不强制，用户可自行添加或移除。

可选：Headroom 上下文压缩

桌面端的运行时由用户自选，使用成本也由用户自选。作为一**加挂**层，Cubecloud 在 `.github/skills/headroom-workflow/` 里附带了一个仓库自带的工作流技能，在本地 token 压力大时（大工具日志、长会话历史、庞大的 CodeGraph 包），会引导 Copilot / VS Code 会话去使用 `headroom-ai` 上下文压缩代理。 在非仓库 Copilot 会话下的完整安装路径 — 包括 `headroom proxy`、`headroom mcp install` 与 `headroom wrap copilot` 三种模式 — 见 `docs/agent-skills-bundle/HEADROOM.md`。 将工作流技能镜像到用户的 Copilot 技能目录的一条命令在 `docs/agent-skills-bundle/install-headroom-workflow.cmd`。 Headroom **永远不是必需的**：即便不安装它，桌面端也完全可用。

概念模型（V2.10.35）

Cubecloud 的本地表面可以干净地映射到一个小**对象 + 动作**模型。这个模型在概念上接近 Palantir Foundry 所说的企业级"本体（Ontology）"，但**范围被收敛到单个操作者的一张桌面上**：

- **对象（名词）** 是智能体操作的持久化事物：人设、会话、模型、提供者、技能、记忆、工具、计划任务与看板任务。它们以显式 JSON 注册表的形式持久化、归本地用户掌控，而不是托管在工作流引擎里的不透明 blob。
- **动作（动词）** 是智能体对这些对象执行的操作：分发一次聊天回合、运行一条计划任务、提交一条学习提案、应用一条 CodeGraph 查询结果，或者回滚一次 `AGENTS.md` 修改。`agentControlPlane.ts` 中的分发流水就是动作日志；`headroom learn --apply` 的复核流就是**分支-复核**闸门，防止 AI 在没有显式人工动作的情况下写入本体。
- **操作者范围** — Foundry 的本体是组织的数字孪生；Cubecloud 的本体是一张桌面的数字孪生。这是一个有意识的尺度差异，不是缺失功能。同样的名词 / 动词结构可以下放到独立开发者与小型企业操作者身上，而不需要让他们为他们并不需要的多租户平台付费。

明确写出这个模型的目的，是让新的贡献者能够把任何未来的特性映射到"这是一个新对象"或"这是一个新动作"，并把它放到架构中正确的层。当前实现仍然使用手写的 JSON 注册表；未来某次硬化重构可以把它们合并成一个强类型的注册表，而不需要改变这份概念契约。

从哪里开始

- **新贡献者**：阅读 `docs/HANDBOOK.md` 第 1、第 2、第 3 与第 5 节。
- **评估桌面端的读者**：先读 `agent-desktop/README.md`，再读 `docs/HANDBOOK.md` 第 1、第 3 与第 10 节。
- **审查者或发布负责人**：按顺序阅读 `docs/HANDBOOK.md` 第 1、第 3、第 4、第 6、第 9、第 10 与第 11 节。

仓库布局

```
cubecloud-agentic-os/
├── README.md                    本单仓 README
├── LICENSE                      Cubecloud 原创部分：AGPL-3.0-or-later /
Apache-2.0 / MIT
├── NOTICE                       第三方归属清单
├── BRANDING_AND_LICENSE.md      许可、来源与版本过渡历史
├── CONTRIBUTING.md              DCO 1.1 贡献契约
```

```
├── SECURITY.md                    安全策略与上报方式
├── THREAT_MODEL.md                本地主导威胁模型
├── README.i18n.md                 译文清单
├── .agents/                       34 个镜像到 ~/.agents/skills/ 的开源技能
├── .github/                       智能体指令、工作流技能与自动化
│   └── skills/
│       └── headroom-workflow/     面向可选 Headroom 代理的 Copilot / VS
Code 工作流层
├── apps/
│   └── desktop-shell/             Cubecloud 原创控制平面工作区
├── packages/
│   └── platform-core/             共享 TypeScript 契约
├── docs/
│   ├── HANDBOOK.md                主手册
│   ├── RETIRED_AND_LEGACY.md      活跃 / 镜像 / 暂存区映射
│   ├── handbook/                  架构、开发、运维等长文
│   └── legal/                     EULA、商标与商业许可政策
├── scripts/
│   ├── sync-docs.ps1              硬链接与目录连接重建脚本
│   └── v2.10.20-readme-combined-pdf.cjs
└── agent-desktop/                 面向用户交付的 Electron 桌面端
```

许可

Cubecloud 原创部分提供 **AGPL-3.0-or-later、Apache-2.0、MIT 三选一。** AGPL-3.0-or-later 是主许可。Apache-2.0 与 MIT 是给下游在机构政策上更容易兼容的选项。 继承的 `hermes-desktop` 框架代码仍按上游 MIT 条款提供。

具体路径级拆分请参见 `LICENSE`、 `NOTICE`、 `BRANDING_AND_LICENSE.md` 与 `docs/legal/`。

贡献

入库贡献遵循 **DCO 1.1** 签名模型。每个提交都必须包含 `Signed-off-by:` 行。 详见 `CONTRIBUTING.md`。

技能层是贡献者的重要工作流表面。新增技能通常要经过 `gbrain-skillify`、 `ecc-skill-scout`、 `po-write-a-skill` 与 `sp-write-skill`，并带有一份 red-baseline 测试来证明这个

行为值得保留。

如果你发现 bug 或有功能请求，请 提交 issue。 安全问题请遵循 `SECURITY.md`，不要在公开 issue 中发布凭据、API 密钥或私人日志。

译文

单仓当前提供以下译文文档：

- `README.zh-CN.md`
- `README.ja-JP.md`
- `README.ko-KR.md`
- `CONTRIBUTING.zh-CN.md`
- `SECURITY.zh-CN.md`
- `THREAT_MODEL.zh-CN.md`
- `docs/HANDBOOK.zh-CN.md`
- `docs/handbook/` 下的 zh-CN 长文
- `docs/RETIRED_AND_LEGACY.zh-CN.md`

译文清单位于 `README.i18n.md`。 英中合并的 README PDF 位于 `docs/Cubecloud-README-en-zh.pdf`。

面向二进制文件的译文仍位于 `agent-desktop/` 下。 如果你想为单仓补充更多语言，或继续润色现有译文，请遵循 `README.i18n.md` 中的流程。